Laboratory work No 1

Combinational logic schemes

Aim of the work

Learn Boolean function minimisation and combinational logic circuit design using low-integrity logic elements and multiplexers and their modelling.

Laboratory work task

The table of problem options shows functions whose argument conjunctions are given as numbers. Each student is given the number of his/her variant. To complete the task:

1. Write the given function in normal disjunctive form.
2. Minimise the given function.
3. This function can be implemented in three ways:
   (a) using AND, OR, NOT elements;
   (b) using only NAND or NOR and NOT elements;
   (c) using a multiplexer and the required AND, OR, NOT, NAND, NOR elements.
4. Verify the functioning of the designed circuits.
5. Prepare a lab report. The report should contain the results of the function minimisation, the implemented schemes and the simulation results of these schemes.

Workflow

1) Using the Karnaugh map template, we load the given function into the chart;
2) Minimise the function and write down the resulting expression or minimise the resulting list of cubes;
3) Restructure the expression by putting letters common to several conjunctions outside the brackets;
4) In Diamond Lattice, use the graphical editor to draw a diagram using AND, OR and NOT elements;
5) Translate the schematic drawing into a description in VHDL;
6) Call the ModelSim wizard;
7) We develop test suites - modelling directives;
8) We simulate the operation of the scheme to check its correctness; if there are errors, we correct them;
9) Repeat steps 4 to 8 for a circuit consisting of NAND gates (and inverters);
10) Do the same for the multiplexer scheme;
11) Prepare a work report.

Notes:

- It is preferable to draw all three schemes on the same page of the graphical editor so that the simulation results can be compared. If necessary, enlarge the sheet.
- The functioning of each diagram needs to be demonstrated to the trainer.

Example of a laboratory arbor

Synthesise the diagrams for a function whose numerical form is:

***F* = (8,14,16,18,19,22,23,24,30,40,42,47,48,50,55,56,58,63).**

The Karnaugh map for this function looks like this:

abc

def	000	001	011	010	110	111	101	100
000		**1**	**1**	**1**	**1**	**1**	**1**	
001								
011				**1**				
010				**1**	**1**	**1**	**1**	
110		**1**	**1**	**1**				
111				**1**	**1**	**1**	**1**	
101								
100								

First merger:

abc

def	000	001	011	010	110	111	101	100
000		**1**	**1**	**1**	**1**	**1**	**1**	
001								
011				**1**				
010				**1**	**1**	**1**	**1**	
110		**1**	**1**	**1**				
111				**1**	**1**	**1**	**1**	
101								
100								

This gives the following cube:

```
abc  def
--1  000
```

Two more mergers:

abc

def	000	001	011	010	110	111	101	100
000		**X**	**X**	**1**	**1**	**X**	**X**	
001								
011				**1**				
010				**1**	**1**	**1**	**1**	
110		**1**	**1**	**1**				
111				**1**	**1**	**1**	**1**	
101								
100								

This produces the following cubes:

```
abc  def
-10  0-0
010  -1-
```

Remaining connections:

abc

def	000	001	011	010	110	111	101	100
000		X	X	X	X	X	X	
001								
011				X				
010				X	X	1	1	
110		1	1	X				
111				X	1	1	1	
101								
100								

This produces the following cubes:

```
abc  def
1-1  010
1-1  111
0-1  110
11-  111
```

What we get is a set of cubes:

```
abc  def  abc  def
--1  000  1-1  111
-10  0-0  0-1  110
010  -1-  11-  111
1-1  010
```

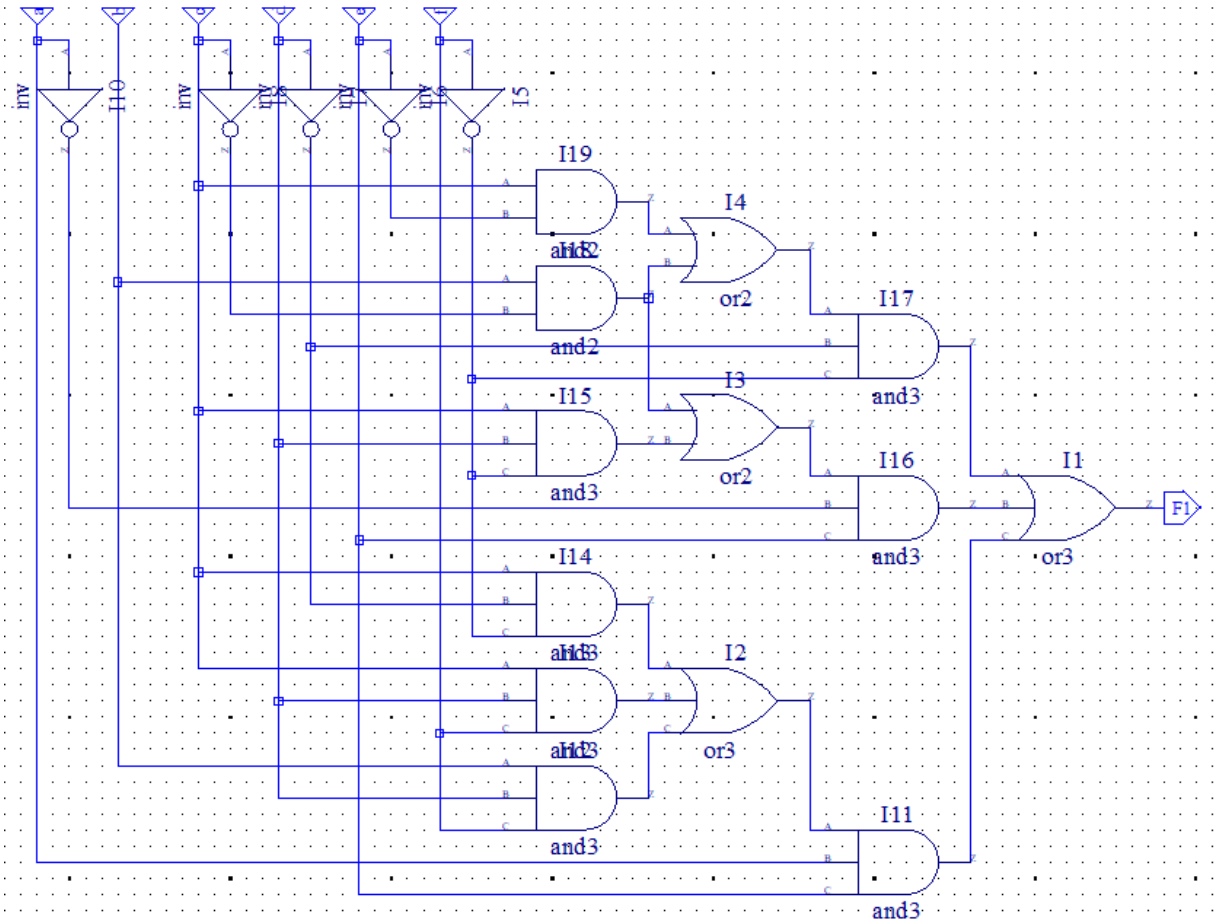
For a parenthetical expression, we use the same colour to highlight the grouped conjunctions, and frames to highlight the common letters:

```
abc  def
--1  000
-10  0-0   - from these conjunctions we derive d̄f̄
010  -1-
0-1  110   - from these conjunctions we raise ā e
1-1  010
1-1  111
11-  111   - from these conjunctions we will raise a e
```

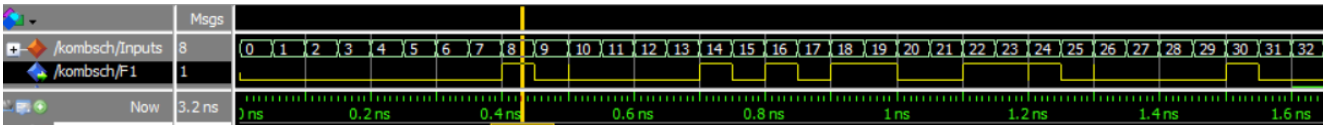
So the parenthetical expression looks like this:

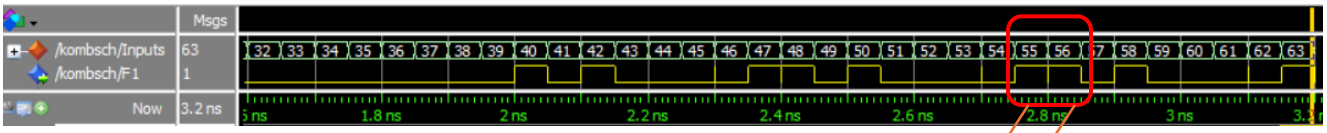
$$F = \bar{d}\bar{f}\,(c\bar{e} + b\bar{c}) + \bar{a}e\,(b\bar{c} + cd\bar{f}) + ae\,(c\bar{d}\bar{f} + cdf + bdf).$$

A diagram drawn with Lattice Diamond's graphical editor using the AND, OR and NOT elements looks like this:



Its simulation results show that the scheme is designed and drawn correctly:

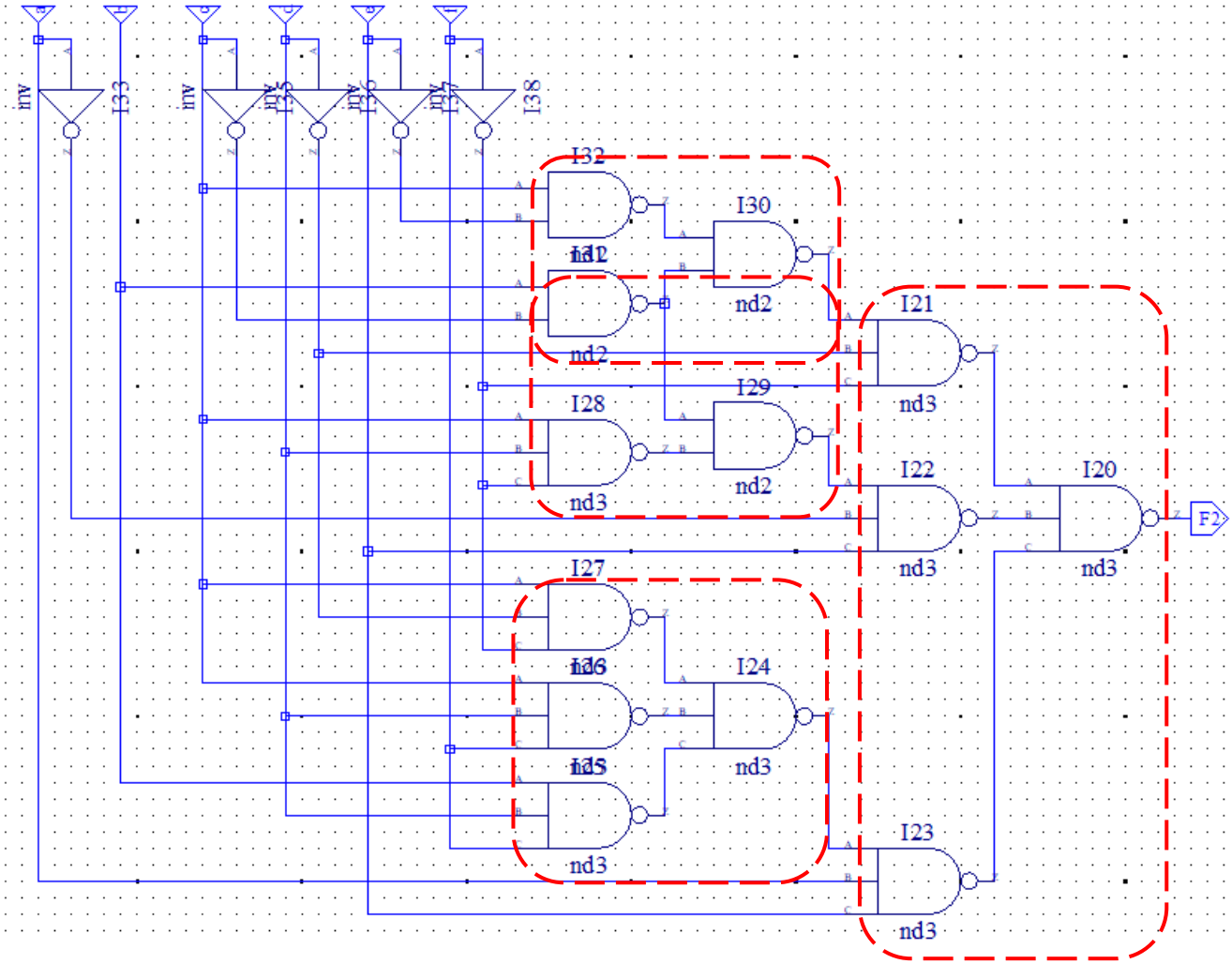




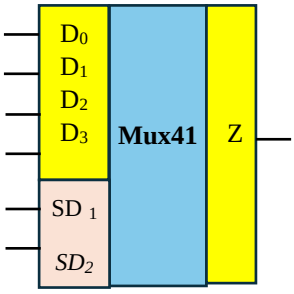
Let's compare:

F **= (8,14,16,18,19,22,23,24,30,40,42,47,48,50,55,56,58,63).**

Using the property that a two-level diagram of AND and OR elements corresponds to a diagram of the same configuration of NAND elements, we can easily draw such a diagram (with the corresponding fragments defined by the red dashed lines):



We will now design a scheme using a 4-1 multiplexer:



Using ***a*** and ***d*** as address variables, the diagram can be split into four parts (separated here by dotted lines, and each part indicates which part the corresponding function diagram is connected to the multiplexer input):

abc

def	000	001	011	010	110	111	101	100
000		1	1	1	1	1	1	
001								
011				1				
010				1	1	1	1	
110		1	1	1				
111				1	1	1	1	
101								
100								

D_0 D_2 D_1 D_3

Let's minimise the functions of each part:

D D_0 2

bc

ef	00	01	11	10
00		1	1	1
01				
11				1
10				1

bc

ef	10	11	01	10
00	1	1	1	
01				
11				
10	1	1	1	

Minimising these functions, we get:

$$F_0 = c\bar{e}\bar{f} + b\bar{e}\bar{f} + b\bar{c}e;$$

$$F_2 = b\bar{f} + c\bar{f} = \bar{f}(b + c).$$

In order to make the configuration of the "1" layout the same as in the 6-variable diagram, the row codes in the 4-variable Karnaugh maps below are in a different order.

D D_1 3

bc

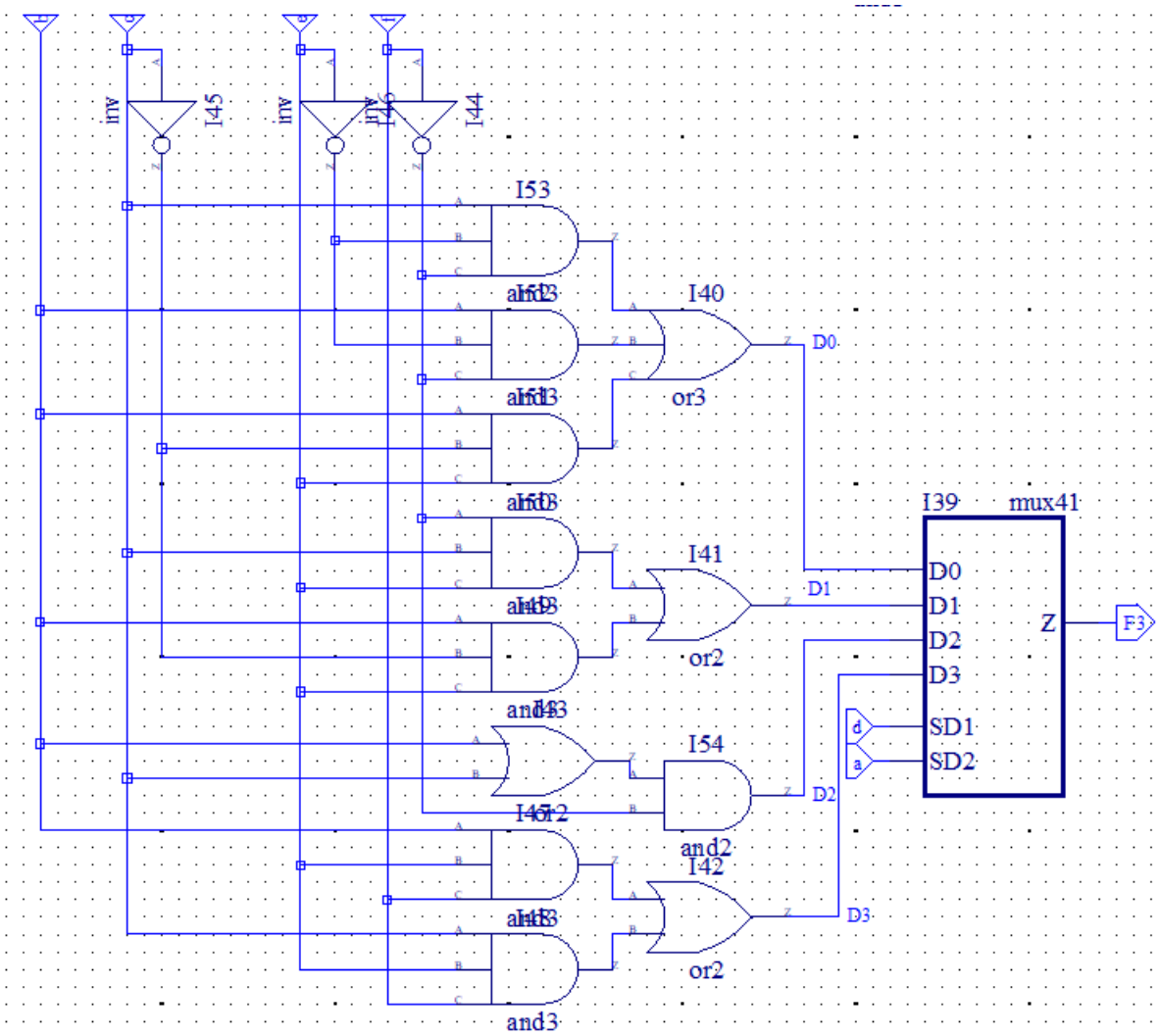
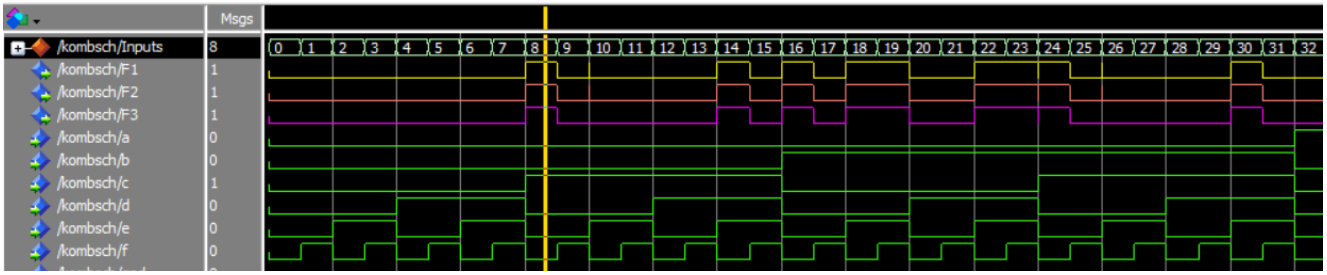
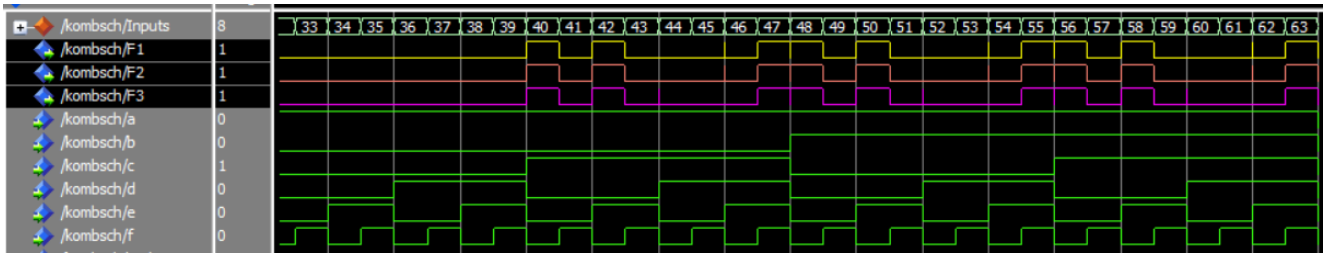
ef	00	01	11	10
10		1	1	1
11				1
01				
00				

bc

ef	10	11	01	10
10				
11	1	1	1	
01				
00				

Minimising the functions, we get:

$$F_1 = ce\bar{f} + b\bar{c}e;$$
$$F_3 = bef + cef.$$



As all the schemes were drawn on the same sheet of the graphical editor, modelling them together ensures that the second and third schemes are also designed and drawn correctly: